A Comprehensive Evaluation of One vs Rest and One vs One Strategies on LeNet, MobileNetV2, and ResNet18 for Multiclass Handwritten Digit Classification

Kartik C
Department of Computer Science (A.I)
KLE Technological University
Email: 01fe23bci107@kletech.ac.in

Goutam K
Department of Computer Science (A.I)
KLE Technological University
Email: 01fe23bci098@kletech.ac.in

Riya P
Department of Computer Science (A.I)
KLE Technological University
Email: 01fe23bci027@kletech.ac.in

Shakti P
Department of Computer Science (A.I)
KLE Technological University
Email: 01fe23bci011@kletech.ac.in

Arundati Aralimatti
Department of Computer Science (A.I)
KLE Technological University
Email: arundati.aralimatti@kletech.ac.in

Veena Hunasimarad
Department of Computer Science (A.I)
KLE Technological University
Email: veena.hunasimarad@kletech.ac.in

***Abstract*—This paper presents the implementation and comparison of One vs Rest (OvR)[19], [7] and One vs One (OvO)[7], [1] multiclass classification using LeNet, MobileNetV2, and ResNet18[12], [16], [18] models on the MNIST dataset[4]. We train 165 required binary classifiers (10 OvR and 45 OvO per backbone) and evaluate training cost, inference speed, prediction consistency, and per-class value across architectures of different capacities. The results presents that OvR matches or does better than OvO while requiring 4 to 5 times fewer computations, with ResNet18 with OvR achieving the highest accuracy of 99.27%. The outputs shows that strong feature extractors [6] benefit from simpler OvR heads, providing good direction for selecting efficient multiclass strategies in deep-learning systems.**

***Index Terms*—One vs Rest, One vs One, Multiclass Classification, Deep Learning, LeNet, MobileNetV2, ResNet18, MNIST**

I. Introduction

Deep Neural Networks is the foundation of modern machine learning systems and is the main part in digit recognition, document analysis, and biometrics [12]. Foundational works in deep learning established the representational capabilities of layered neural models [6], [13], [20], while early convolution-based architectures demonstrated the effectiveness of translation-invariant feature extractors for image understanding [12], [11], [21], [18]. Subsequent advancements in activation functions [17], optimization algorithms [10], [23], and normalization techniques [9] improved training stability and scalability. Efficient architectures such as MobileNetV2 [16] and EfficientNet [22] further extended the applicability of deep models to resource-constrained settings, while visualization-driven studies highlighted the interpretability of learned representations [24].

Multiclass image classification remains a important problem in machine learning, where the mapping from high-dimensional embeddings to separate labels is done through classification heads. Research in Statistical Learning Theory showed binary reduction strategies to solve this problem in kernel based models, most important One vs Rest (OvR) and One vs One (OvO) formulations. These methods gave key insights into margin based separability [3], [7], class interaction behavior [5], and error-correcting output code structures [19], [1].

The rise of Convolutional Neural Networks (CNNs) [12], [11] has reshaped the working of multiclass classification techniques. Modern CNNs such as LeNet, MobileNetV2, and ResNet18 give feature embeddings with better intraclass compactness and interclass separability [6]. In combination with well structured datasets like MNIST [4], [14], this raises important questions about the computational and predictive differences between OvR and OvO when applied to contemporary deep architectures.

This work presents comparison of OvR and OvO across three representative CNN backbones LeNet (shallow and classical), MobileNetV2 (lightweight and efficient), and ResNet18 (deep residual). A complete set of 165 binary classifiers (10 OvR and 45 OvO per architecture) is trained for better evaluation. The analysis includes training duration, inference latency, class wise pattern, prediction stability, and computational overhead. The results show that OvR consistently matches or surpasses OvO while requiring fewer computations. ResNet18 with OvR achieves the highest accuracy of 99.27%, showcasing the effectiveness of strong feature extractors along with simpler classification heads [6].

The remainder of this paper is organized as follows. Section II reviews related work on multiclass reduction and CNN architectures. Section III describes the architectures and methodology used in the study. Section IV presents the

experimental results, including accuracy, inference time, and per-class analyses. Section V provides concluding remarks.

II. Related Work

Multiclass classification methods have become an important in both statistical learning and neural network research. Early work in statistical learning introduced the OvR and OvO schemes as practical methods for extending binary classifiers to multiclass problems. OvR trains a single classifier each class, while OvO makes a classifier for every pair of classes. These methods were extensively tested in the context of Support Vector Machines (SVMs), where margin behavior, class interactions, and computational implications were analyzed [3], [7]. Extensions to these strategies were explored through polychotomous classification [5], unified margin formulations [1], and Error-Correcting Output Codes (ECOC) that improve robustness against classifier-level errors [19]. These studies provided theoretical and practical foundations that continue to influence multiclass reduction design choices.

The emergence of deep learning made important changes in multiclass classification. Convolutional Neural Networks (CNNs) learn hierarchical visual representations that presents strong class separation and reduced overlap in embedding space [6], [13]. Foundational architectures such as LeNet [12], VGG [21], and AlexNet [11] demonstrated the advantages of convolutional feature extraction for image recognition. Subsequent advancements in activation functions [17], optimization algorithms [10], [23], and normalization techniques [9] further stabilized training and enhanced scalability. Modern CNN families such as MobileNetV2 [16] and EfficientNet [22] achieved high performance with significantly reduced computational cost, enabling deployment in mobile and embedded systems. Deeper residual architectures like ResNet18 [18] improved feature learning through identity based skip connections, addressing gradient degradation in deep networks.

LeNet-5 is one of the earliest CNNs and consists of two convolution– subsampling stages followed by fully connected layers. Its simplicity, low computational cost, and well-structured hierarchy make it a standard baseline in digit recognition and lightweight deep learning studies.

MobileNetV2 introduces inverted residual blocks with linear bottlenecks, combining expansion, depthwise convolution, and

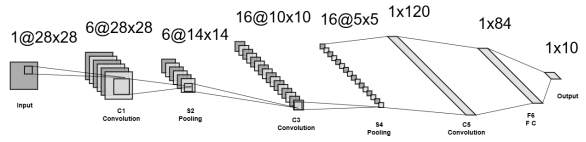

Fig. 1: LeNet-5 architecture illustrating the convolution–subsampling hierarchy [12].

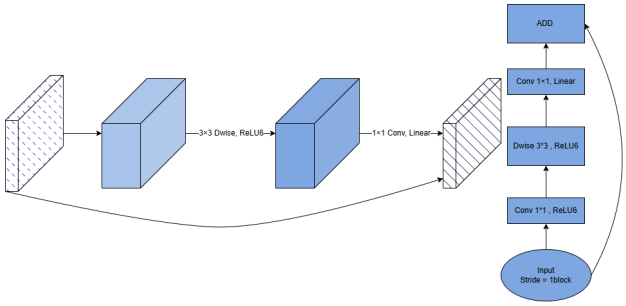

Fig. 2: Inverted residual block with linear bottleneck in MobileNetV2 [16].

projection layers. This design significantly reduces computation while preserving feature quality, making MobileNetV2 suited for mobile and embedded inference.

ResNet-18 is constructed using a sequence of BasicBlocks, each implementing the residual mapping shown in Fig. 3. Every block uses two convolutional layers to compute a transformation $F(x)$, which is added to the identity shortcut x, producing the output $F(x) + x$ [18]. This skip-connection mechanism mitigates vanishing gradients and enables stable training in deeper networks, making ResNet-18 a widely adopted backbone for image classification.

Research comparing OvR and OvO in deep feature spaces has observed that strong embeddings reduce the need for numerous pairwise classifiers. Rifkin and Klautau [19] showed that OvR can match or exceed the performance of OvO in many settings, especially when class separability is high. Meanwhile, OvO remains attractive in scenarios where fine-grained pair interactions are crucial, though at the cost of substantially increased training and inference computation [7]. Studies on prediction calibration, voting mechanisms, and decision consistency highlight that pairwise voting in OvO can introduce instability as the number of class pairs grows [19], [7].

Existing literature provides strong theoretical and empirical foundations, but relatively few works offer a fully imple-

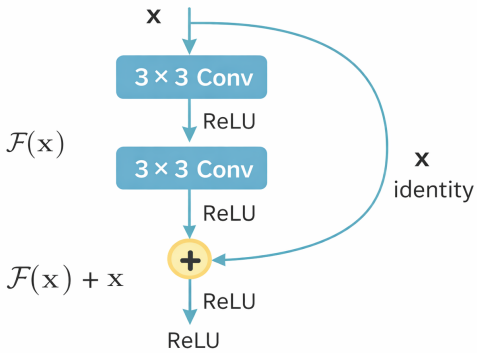

Fig. 3: Basic residual block used in ResNet-18 and ResNet-34, illustrating identity skip connections [18].

mented, architecture-sensitive comparison of OvR and OvO across CNN backbones of varying depth and efficiency. This gap motivates the present study, which evaluates both strategies using LeNet, MobileNetV2, and ResNet18 and reports detailed measurements of accuracy, inference cost, and class-wise performance.

III. Methodology

This section describes the complete experimental pipeline to evaluate One-vs-Rest and One-vs-One strategies across LeNet, MobileNetV2, and ResNet18 architectures. The methodology covers multiclass decomposition, binary classifier formulation, loss functions, training configurations, and inference procedures. Care was taken to ensure that all implementations remain architecture-aware so that comparisons between OvR and OvO are fair and reproducible.

A. Multiclass Reduction Strategies

1) One-vs-Rest (OvR): For each class c, a binary classifier is trained to distinguish class c from all other classes [19], [5]. The binary label is defined as shown in Eq. 1:

$$y^{(c)} = \begin{cases} 1, & y = c, \\ 0, & y \neq c. \end{cases} \tag{1}$$

Here, $y^{(c)}$ denotes the binary target for class c, where samples belonging to class c are labeled 1 and all remaining classes are labeled 0.

Each classifier outputs a logit z_c, which is converted to a probability via the sigmoid[2] as depicted in Eq. 2:

$$f^{(c)}(\tilde{x}) = \sigma(z_c). \tag{2}$$

In this expression, $f^{(c)}(\tilde{x})$ represents the predicted probability for class c, obtained by applying the sigmoid to the classifier logit z_c.

The predicted class is obtained by selecting the classifier with the highest output probability[19] as defined in Eq. 3:

$$\hat{y} = \arg\max_c f^{(c)}(\tilde{x}). \tag{3}$$

Here, $\hat{y}$ denotes the final OvR prediction, chosen as the class index whose binary classifier outputs the maximum probability.

For 10 classes, the number of OvR classifiers is expressed as Eq. 4:

$$N_{\text{OvR}} = 10. \tag{4}$$

Here, N_{OvR} denotes the total number of binary classifiers required in the One-vs-Rest strategy, which equals the number of classes.

2) One-vs-One (OvO): In OvO, a separate binary classifier is trained for every unordered pair (i, j) of classes[7], [1] as described in Eq. 5:

$$y^{(i,j)} = \begin{cases} 1, & y = i, \\ 0, & y = j. \end{cases} \tag{5}$$

Here, $y^{(i,j)}$ is the binary target indicating whether a sample belongs to class i (label 1) or class j (label 0), used in each pairwise classifier.

The total number of pairwise classifiers for 10 classes is shown as Eq. 6:

$$N_{\text{OvO}} = \binom{10}{2} = 45. \tag{6}$$

Here, N_{OvO} denotes the total number of binary classifiers required in the One-vs-One strategy, obtained by selecting all unordered pairs of classes, which results in 45 classifiers for MNIST.

Inference is performed using majority voting[7]. Each pairwise classifier casts one vote for either class i or class j, and the final prediction is represented as Eq. 7:

$$\hat{y} = \arg\max_c \sum_{i<j} \mathbf{1}\Big(f^{(i,j)}(\tilde{x}) = c\Big) \tag{7}$$

Here, the predicted label $\hat{y}$ is the class receiving the highest number of votes across all pairwise decisions.

B. Binary Loss Function

Each binary classifier is trained with binary cross-entropy using logits[2] as formulated in Eq. 8:

$$\mathcal{L}(z, y) = \max(z, 0) - zy + \ln(1 + e^{-|z|}), \tag{8}$$

In this loss, z is the model logit, $y \in \{0, 1\}$ is the binary label, and $\mathcal{L}(z, y)$ measures the penalty for mismatching predictions and targets.

Gradient is defined as Eq. 9:

$$\frac{\partial \mathcal{L}}{\partial z} = \sigma(z) - y. \tag{9}$$

This gradient shows that the update is driven by the difference between the predicted probability $\sigma(z)$ and the true label y[2].

C. Proposed Work

This work provides a detailed, reproducible evaluation of OvR and OvO applied to three representative CNN backbones[12], [16], [18] that span shallow, efficient, and deep design points. The experimental pipeline trains all required Binary Classifiers for each decomposition strategy to ensure a controlled comparison. The evaluation measures training duration, inference latency, prediction consistency[7], [19], and per-class performance, and it quantifies computational cost differences between strategies. The study reports that ResNet18 combined with OvR attains the highest accuracy observed in our experiments and documents computational savings of approximately four to five times[7] relative to OvO

Algorithm 1: ResNet Forward Pass [18]

Input: Input image x
Output: Predicted class $\hat{y}$

```
x_0 ← Conv-BN-ReLU(x)                    // Stem layer
foreach residual block i do
    y ← F(x_i)                           // Convolutional sub-block
    if dim(y) = dim(x_i) then
        x_{i+1} ← y + x_i                // Identity skip connection
    else
        x_{i+1} ← y + Proj(x_i)          // 1 × 1 projection skip
z ← GlobalAvgPool(x_final)
ŷ ← Softmax(FC(z))
return ŷ
```

Algorithm 2: LeNet-5 Forward Propagation[12]

Input: Input image $x \in \mathbb{R}^{32\times 32}$
Output: Predicted class $\hat{y}$

```
h_1 ← Conv_{5×5}(x)                      // C1
h_2 ← AvgPool_{2×2}(h_1)                 // S2
h_3 ← Conv_{5×5}(h_2)                    // C3
h_4 ← AvgPool_{2×2}(h_3)                 // S4
h_5 ← Conv_{5×5}(h_4)                    // C5
f ← Flatten(h_5)  h_6 ← ReLU(W_6 f + b_6)   // F6
h_7 ← ReLU(W_7 h_6 + b_7)                // F7
ŷ ← Softmax(W_8 h_7 + b_8)               // Output layer
return ŷ
```

across the tested backbones. These contributions aim to guide the selection of efficient multiclass reduction strategies for practical deep-learning deployments.

To ensure architecture-aware evaluation across LeNet, MobileNetV2, and ResNet18, the exact forward propagation algorithms used during experimentation are provided here.

D. Training Configuration

LeNet-5 is trained from scratch using SGD with momentum on the MNIST training split. MobileNetV2 and ResNet18 are initialized from ImageNet-pretrained weights. For MobileNetV2, the convolutional backbone is frozen and only a single-logit binary head is trained for each OvR/OvO classifier. ResNet18 is fine-tuned end-to-end. All binary classifiers are trained with BCE-with-logits loss, mini-batch training on GPU, and fixed random seeds for reproducibility.

E. Inference Pipeline

OvR requires evaluating 10 classifiers; OvO requires evaluating 45 pairwise classifiers followed by vote aggregation. For timing measurements, inference was run on batch-processed test data.

Algorithm 3: MobileNet Forward Propagation (Depthwise Separable CNN)[16]

Input: Input image $x \in \mathbb{R}^{H\times W\times 3}$
Output: Predicted class $\hat{y}$

```
h_0 ← Conv_{3×3,s=2}(x)                  // Initial Conv + BN + ReLU
for i = 1 to L do
    h_dw ← DWConv_{3×3}(h_{i-1})         // Depthwise conv
    h_pw ← PWConv_{1×1}(h_dw)            // Pointwise conv
    h_i ← BN+ReLU(h_pw)                  // Activation
z ← GlobalAvgPool(h_L)                   // Feature aggregation
o ← Wz + b                               // Fully connected layer
ŷ ← Softmax(o)                           // Prediction
return ŷ
```

F. Reproducibility

All training logs, model checkpoints, and evaluation outputs were consistently saved with timestamped identifiers to ensure complete experimental reproducibility.

IV. Results

This section presents a comprehensive evaluation of the OvR and OvO classification strategies across LeNet-5, MobileNetV2, and ResNet-18. The analysis covers overall accuracy, macro-level precision, recall, F1-score, inference latency, per class performance, and prediction distribution, based on the updated experimental implementation.

A. Dataset Description

The MNIST dataset consists of 60,000 training and 10,000 testing samples[4], [14], each represented as a 28×28 grayscale digit. All experiments are performed on the standard training and test splits without any data augmentation.

B. Data Preprocessing

Each backbone uses a preprocessing pipeline that matches the requirements of its architecture and the exact behavior of the training scripts. LeNet-5 operates on inputs resized to 32×32[12](scaling pixel values to [0,1]) and normalized

TABLE I: Final Performance Metrics Across Models (OvR vs OvO). Total inference time is measured on the full 10k MNIST test set.

Model	Strategy	Acc	F1	Total Inference Time (s)
LeNet	OvR	0.9862	0.9861	31.74
	OvO	0.9803	0.9802	137.95
MobileNetV2	OvR	0.9741	0.9737	5.43
	OvO	0.9518	0.9508	23.97
ResNet18	OvR	0.9927	0.9926	37.07
	OvO	0.9847	0.9844	151.93

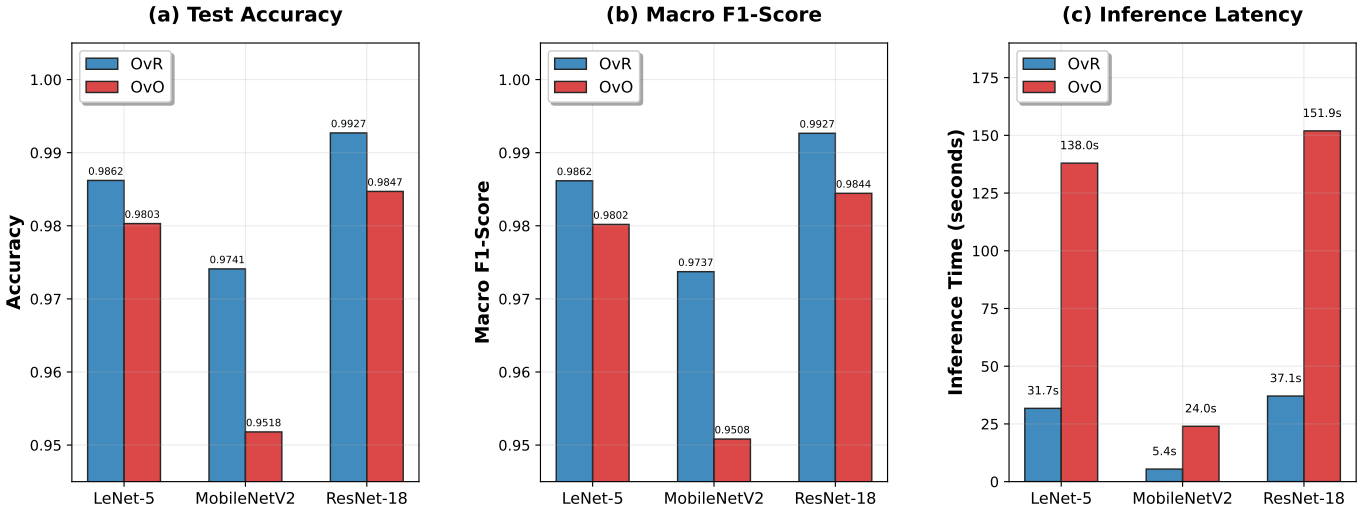

Fig. 4: Cross-architecture comparison of accuracy, macro F1-score, and inference latency for OvR and OvO.

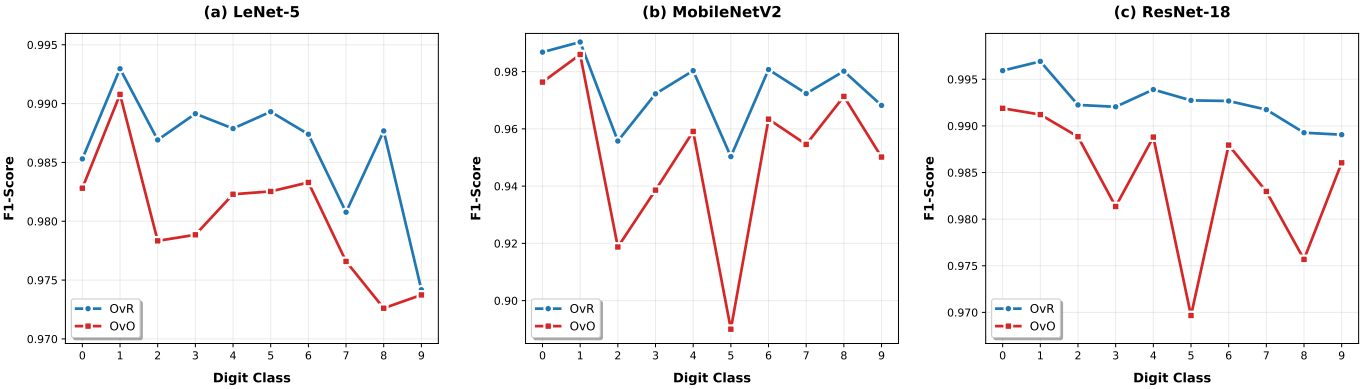

Fig. 5: Per-class F1-score comparison for OvR and OvO across LeNet, MobileNetV2, and ResNet-18.

using a simple grayscale normalization. MobileNetV2 and ResNet18 operate on inputs resized to 224×224[16], [18]. Since MNIST images are originally grayscale, the single channel is repeated three times to form an RGB input. No ImageNet-style mean–std normalization is applied; the models use the same lightweight normalization used during training.

C. Overall Quantitative Performance

Table I summarizes the accuracy, macro F1 score, and inference time for all six configurations evaluated in this study. Across all three backbones, One-vs-Rest (OvR) consistently matches or outperforms One-vs-One (OvO)[19] while also providing substantially faster inference. The largest gap appears in MobileNetV2, where OvO loses more than 2% accuracy and requires over 4 times the inference time[7] due to the evaluation of 45 pairwise classifiers. ResNet18 combined with OvR attains the highest overall performance, achieving an accuracy of 99.27%.

D. Accuracy and Inference Time Trends

The accuracy and latency plots in Fig. 4 show that OvR consistently outperforms OvO while being far more efficient. OvO causes major slowdowns—over fourfold in LeNet and MobileNetV2, and similarly in ResNet18—due to the need to evaluate forty-five pairwise classifiers. This highlights the significant computational inefficiency of the OvO strategy.

E. Per-Class F1 Score Analysis

Fig. 5 shows the per-class F1 trends across the three architectures. OvR yields smoother and more stable performance, including on challenging digits such as 3, 5, and 9 [4]. OvO exhibits larger fluctuations [7], [19] due to inconsistencies in pairwise voting. Stronger feature extractors like ResNet18 reduce this gap, but OvR still provides the more uniform class-wise response.

F. Prediction Distribution and Stability

The prediction distributions in Fig. 6 show clear stability differences between the two strategies. OvR follows the true digit distribution more closely, while OvO tends to overestimate digits like 1 and 7 and underpredict digits such as 3 and 5. These deviations arise from the pairwise voting mechanism, where small classifier imbalances accumulate across the forty-five binary decisions [7].

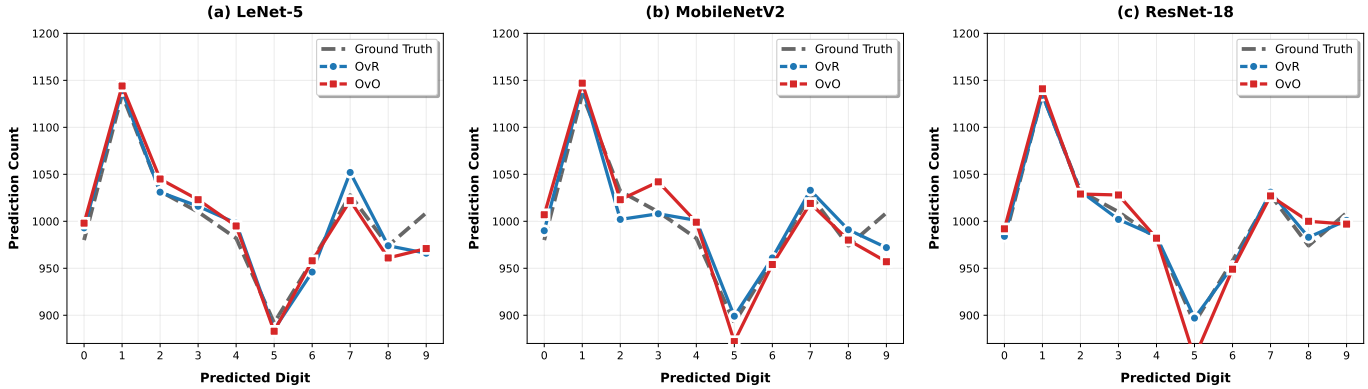

Fig. 6: Prediction distribution of OvR and OvO compared against the ground truth across the three architectures.

G. Cross-Architecture Interpretation

The combined results show a consistent trend across all three networks: OvR delivers higher accuracy, smoother per-class behavior, and far lower computational cost. MobileNetV2 shows the largest gap, with OvO losing accuracy and becoming much slower. ResNet18 benefits from strong deep feature representations [18], enabling OvR to achieve the highest accuracy while avoiding the overhead of evaluating forty-five pairwise classifiers. Overall, OvR proves to be the more robust and efficient multiclass strategy across all tested architectures [19].

V. Conclusion

We compared One-vs-Rest (OvR) and One-vs-One (OvO) multiclass reduction strategies in all three CNN backbones LeNet, MobileNetV2, ResNet18 using a total of 165 binary classifiers trained on MNIST. OvR consistently matched or exceeded OvO [19] in accuracy while requiring far less computation, since only 10 classifiers are evaluated instead of 45. This resulted in a four-to-five-fold reduction in inference cost across architectures. ResNet18 with OvR achieved the highest accuracy (99.27%), showing that strong feature extractors can make pairwise decomposition unnecessary in well-separated embedding spaces [6]. In contrast, OvO introduced instability in the voting process [7] and exhibited larger per-class variance, especially when the backbone was frozen or lightly optimized. Overall, OvR emerges as the more efficient and reliable strategy for modern CNN embeddings on compact datasets such as MNIST.

References

[1] E. Allwein, R. E. Schapire, and Y. Singer, "Reducing Multiclass to Binary: A Unifying Approach for Margin Classifiers," *Journal of Machine Learning Research*, vol. 1, pp. 113–141, 2000.

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[3] C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[4] L. Deng, "The MNIST Database of Handwritten Digit Images for Machine Learning Research," *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.

[5] J. H. Friedman, "Another Approach to Polychotomous Classification," *Technical Report*, Stanford University, 2000.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[7] C.-W. Hsu and C.-J. Lin, "A Comparison of Methods for Multiclass Support Vector Machines," *IEEE Transactions on Neural Networks*, vol. 13, no. 2, pp. 415–425, 2002.

[8] K. He, R. Girshick, and P. Dollár, "Bag of Tricks for Image Classification with Convolutional Neural Networks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 558–567.

[9] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015.

[10] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[11] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2012, pp. 1097–1105.

[12] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[13] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, pp. 436–444, 2015.

[14] P. Y. Simard, D. Steinkraus, and J. C. Platt, "Best Practices for Convolutional Neural Networks Applied to Visual Document Analysis," in *Proc. Int. Conf. Document Analysis and Recognition (ICDAR)*, 2003.

[15] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollar, "Focal Loss for Dense Object Detection," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2017.

[16] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[17] V. Nair and G. E. Hinton, "Rectified Linear Units Improve Restricted Boltzmann Machines," in *Proc. Int. Conf. Machine Learning (ICML)*, 2010.

[18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[19] R. Rifkin and A. Klautau, "In Defense of One-vs-All Classification," *Journal of Machine Learning Research*, vol. 5, pp. 101–141, 2004.

[20] J. Schmidhuber, "Deep Learning in Neural Networks: An Overview," *Neural Networks*, vol. 61, pp. 85–117, 2015.

[21] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2015.

[22] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. Int. Conf. Machine Learning (ICML)*, 2019.

[23] A. C. Wilson *et al.*, "The Marginal Value of Adaptive Gradient Methods in Machine Learning," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[24] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning Deep Features for Discriminative Localization," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016.